DevPulse API

Software Architecture Document (SAD)

Version: 1.0
Status: Approved
Document Type: Software Architecture Document (SAD)
Related Documents: - 00-project-overview.md - 01-requirements.md - 03-domain-model.md - 11-decisions.md

---

1. Purpose

تصف هذه الوثيقة المعمارية الرسمية لمشروع **DevPulse API.** وتوضح المبادئ الهندسية، بنية النظام، الطبقات، ومسؤولية كل مكون.

أي قرار معماري يجب أن يكون متوافقًا مع هذه الوثيقة.

---

2. Architectural Goals

تم تصميم المعمارية لتحقيق الأهداف التالية:

- سهولة التطوير.
- سهولة الاختبار.
- سهولة الصيانة.
- قابلية التوسع.
- انخفاض الترابط (Low Coupling).
- ارتفاع التماسك (High Cohesion).
- وضوح المسؤوليات.

---

3. Architectural Style

Selected Style

Modular Monolith

Why Modular Monolith?

- مناسب لحجم المشروع الحالي.
- جميع أجزاء النظام داخل Codebase واحد.
- يقلل تعقيد النشر.
- يسمح بفصل الوحدات منطقيًا.

- يمكن مستقبلاً استخراج أي Module إلى Microservice.

Why NOT Microservices?

في الإصدار الأول:

- لا توجد حاجة تقنية.
- يزيد التعقيد.
- يزيد تكلفة التطوير.
- يزيد تكلفة الاختبارات.

---

4. System Context

```
                +------------------+
                |  Next.js Client  |
                +------------------+
                         |
                         |
                +------------------+
                | Flutter Client   |
                +------------------+
                         |
                    HTTPS / JSON
                         |
                +------------------+
                |  Laravel API     |
                +------------------+
                  |      |       |
                  |      |       |
               MySQL   Storage  Cache*
                            |
                       Local / S3
```

* Cache لاحقاً

---

5. Request Lifecycle

كل Request داخل النظام يجب أن يمر بالمراحل التالية:

```
Client

↓
```

```
Route
↓
Middleware
↓
Form Request
↓
Policy (عند الحاجة)
↓
Controller
↓
Service
↓
Model
↓
Database
↓
API Resource
↓
JSON Response
```

أي تجاوز لهذا التسلسل يعتبر مخالفة للمعمارية.

---

6. Layer Responsibilities

Routes

المسؤولية:

- تعريف Endpoints.
- Versioning.

- ربط Controller.

ممنوع:

- كتابة أي Logic.

Middleware

المسؤولية:

- Authentication.
- Rate Limiting.
- CORS.
- Request Filtering.

Form Requests

المسؤولية:

- Validation.
- Authorization البسيطة.
- تنظيف البيانات.

Controllers

المبدأ الأساشي:

Thin Controllers

المسؤوليات:

- استقبال الطلب.
- استدعاء Service.
- إعادة Response.

ممنوع:

- SQL.
- Business Logic.
- Upload Logic.
- Notifications.
- Queries المعقدة.

Services

القلب الحقيقي للنظام.

توضع هنا Business Rules جميع.

أمثلة:

- PostService
- UserService
- AuthService
- CommentService

Models

المسؤوليات:

- العلاقات.
- Scopes.
- Accessors.
- Mutators.

ممنوع:

Business Logic الخاص بالتطبيق.

API Resources

المسؤولية:

- توحيد Responses.
- إخفاء البيانات الحساسة.
- المحافظة على Contract ثابت مع العملاء.

---

7. Module Boundaries

يتكون المشروع من الوحدات التالية:

- Authentication
- Users
- Posts
- Categories
- Tags
- Comments
- Media

كل Module يجب أن يكون مستقلاً قدر الإمكان.

---

8. Dependency Rules

يسمح فقط بالتبعيات التالية:

```
Controller
      ↓
Service
      ↓
Model
      ↓
Database
```

ولا يسمح بالعكس.

---

9. Coding Principles

يلتزم المشروع بالمبادئ التالية:

- SOLID
- DRY
- KISS
- Convention over Configuration
- Explicit is better than Implicit

---

10. Error Handling Strategy

جميع الأخطاء تعاد بصيغة موحدة.

مثال:

```
{
    "success": false,
    "message": "Validation failed.",
    "errors": {
        "title": [
            "The title field is required."
        ]
    }
}
```

---

11. API Response Strategy

جميع الاستجابات تمر عبر:

API Resources

ولا يسمح بإرجاع Models مباشرة.

---

12. Security Architecture

يعتمد النظام على:

- Laravel Sanctum.
- Policies.
- Validation.
- Rate Limiting.
- Secure File Upload.
- Environment Variables.
- Hashing.

---

13. Performance Strategy

الإصدار الأول يعتمد:

- Eager Loading.
- Pagination.
- Query Optimization.
- Database Indexes.

لاحقاً:

- Redis Cache.
- Queues.
- Horizon.

---

14. Scalability Strategy

يمكن إضافة:

- Notifications
- Likes
- Followers

- Bookmarks
- Search Engine
- Reactions
- Analytics

دون إعادة تصميم النظام.

---

15. Architecture Constraints

يلتزم المشروع بالقيود التالية:

- REST فقط.
- JSON فقط.
- Stateless Authentication.
- Laravel Framework.
- عدم استخدام Packages دون مبرر.

---

16. Definition of Done (Architecture)

أي Feature تعتبر مكتملة فقط إذا:

- التصميم واضح.
- Migration مكتملة.
- العلاقات صحيحة.
- Validation موجود.
- Authorization موجود.
- Service موجود.
- Resource موجود.
- Feature Test موجود.
- Documentation محدثة.

---

17. Future Evolution

يمكن مستقبلاً:

- استخدام Redis.
- استخدام Horizon.
- استخدام S3.
- استخراج Services مستقلة.
- اعتماد CQRS عند الحاجة.

---

18. Architecture Review Checklist

قبل دمج أي Feature:

- هل اتبع Layer المناسب؟
- هل يوجد Business Logic داخل Controller؟
- هل استخدم Form Request؟
- هل يوجد Resource؟
- هل يوجد Policy؟
- هل يوجد Test؟
- هل تم تحديث الوثائق؟

---

19. Revision History

Version	Date	Description
1.0	2026-07-21	Initial Architecture